Neural Networks 206 (2027) 109803

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Transductive learning via validation risk minimization for semi-supervised learning

Yan Lv, Zhen Wang, Lan Bai *

School of Mathematical Sciences, Inner Mongolia University, Hohhot, 010030, PR China

ARTICLE INFO

Keywords:
Semi-supervised learning
Transductive learning
Validation risk minimization
Least squares support vector machine
Bilevel programming
Integer programming

ABSTRACT

Transductive learning aims to recover the labels of unlabeled samples in a dataset with several labeled samples. The empirical risk minimization principle is commonly adopted in transductive learning. In this paper, the validation risk minimization principle is proposed and applied in transductive learning. The key idea is to introduce the validation strategies of supervised learning into transductive learning, leading to two learning frameworks with a general learning machine. Generally, these frameworks result in bilevel programming problems. Further, by using the least squares support vector machine as the learning machine in the two frameworks, we propose two transductive learning models which degenerate the bilevel programming problems into linear or quadratic integer programming problems. New algorithms are respectively designed to solve these problems efficiently. Preliminary experiments on synthetic, benchmark, and real-world datasets show that the proposed models achieve competitive transductive performance.

1. Introduction

Transductive learning (Boudiaf et al., 2020; Chapelle et al., 1999; Joachims, 1999; Li et al., 2018; Vapnik, 1998) aims to infer the labels of unlabeled samples in a dataset with several labeled samples. Unlike supervised or some semi-supervised learning (Ding et al., 2017; Hornik et al., 1989; Yang et al., 2022) that focuses on generalization to unseen data, transductive learning focuses on making accurate predictions for the given unlabeled samples. Particularly suited to scenarios in which the samples to be predicted are known in advance, it has been widely applied in fields such as mineral exploration (Hajihosseinlou et al., 2025; Luo et al., 2026; Xu & Zuo, 2024), text classification (Joachims, 1999; Poon et al., 2019; Yao et al., 2019), bioinformatics (Ciampi et al., 2026; Kipf & Welling, 2017; Liang et al., 2023), medical image analysis (Berenguer et al., 2024; Boudiaf et al., 2020; Huang et al., 2025), equipment fault diagnosis (Wu et al., 2024; Zhang et al., 2021; Zhao et al., 2024), etc.

Given labeled and unlabeled samples, most existing transductive learning models follow the empirical risk minimization paradigm (Vapnik, 1998) on the labeled samples and impose geometric or structural constraints on the unlabeled samples (Belkin et al., 2006; Kipf & Welling, 2017). For instance, transductive support vector machines (TSVMs) (Cevikalp & Franc, 2017; Collobert et al., 2006; Joachims, 1999; Singla et al., 2021) jointly optimize the decision boundary and the candidate labels by minimizing the empirical risk over both labeled and unlabeled data, guided by the low-density separation assumption (Chapelle & Zien, 2005). However, when the labeled samples are limited or the prior assumptions deviate from the ground truth, this empirical risk minimization paradigm becomes highly unreliable (Li & Zhou, 2014; Oliver et al., 2018). Fig. 1 shows a toy counterexample consisting of eight samples, of which the two labeled samples belong to different classes.

Every TSVM variant places the decision boundary in the low-density region between the two groups, achieving zero empirical risk on the labeled samples; the boundary in Fig. 1b is therefore representative. Nevertheless, it misclassifies samples that should be positive. Recently, deep learning has been extensively explored in transductive learning, with approaches including graph neural networks (Kipf & Welling, 2017; Shi et al., 2025; Wang & Liu, 2025), pseudo-label propagation (Iscen et al., 2019), and transductive few-shot learning (Liu et al., 2019). However, existing deep learning approaches remain sensitive to pseudo-label noise and to distribution mismatch (Arazo et al., 2020; Oliver et al., 2018).

Since the labeled samples are too few for the empirical risk to serve as a reliable estimate of the expected risk (Vapnik, 1998), we propose a new framework called validation risk minimization in this paper. Inspired by validation strategies in machine learning, we minimize the validation error during learning rather than the empirical risk. Traditionally, there are two strategies to validate a learning model, i.e., using a validation set or k-fold cross-validation (Bengio & Grandvalet, 2004). Therefore, we propose a transductive learning framework based on a validation set (TLFVS) and another based on cross-validation (TLFCV).

* Corresponding author.
E-mail address: imubailan@163.com (L. Bai).

https://doi.org/10.1016/j.neunet.2026.109803
Received 20 June 2026; Received in revised form 7 September 2026; Accepted 24 October 2026
Available online 26 October 2026
0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.



(a) Ground-truth (b) TSVMs' Prediction

Fig. 1. A toy example of TSVMs for transductive learning. (a) The ground-truth classes, positive on the left and negative on the right. (b) The prediction by a TSVM with its decision.

Both TLFVS and TLFCV employ a general learning machine to infer the labels of unlabeled samples, and are formulated as bilevel programming problems (Blondel et al., 2022; Chen et al., 2024; Petrulionyte et al., 2024). Their risk bounds are also discussed theoretically. To validate their feasibility, the least squares support vector machine (LSSVM) (Suykens & Vandewalle, 1999) is employed in TLFVS and TLFCV as the learner, yielding $\text{TLFVS}_{\text{LSSVM}}$ and $\text{TLFCV}_{\text{LSSVM}}$. Since LSSVM has a closed-form solution, $\text{TLFVS}_{\text{LSSVM}}$ reduces to a linear integer programming problem that is solved efficiently, and $\text{TLFCV}_{\text{LSSVM}}$ reduces to an NP-hard quadratic integer programming problem. To solve this NP-hard problem, a fast greedy algorithm is designed and its finite step termination is guaranteed. The main contributions of this paper are summarized as follows:

(i) Validation risk minimization is proposed for transductive learning, leading to two frameworks TLFVS and TLFCV;
(ii) As an example, the LSSVM is employed as the learning machine in TLFVS and TLFCV to construct two transductive learning models ($\text{TLFVS}_{\text{LSSVM}}$ and $\text{TLFCV}_{\text{LSSVM}}$);
(iii) The linear integer programming problem in $\text{TLFVS}_{\text{LSSVM}}$ and the NP-hard quadratic integer programming problem in $\text{TLFCV}_{\text{LSSVM}}$ are solved efficiently by proposing two algorithms, respectively;
(iv) The experimental results on synthetic, benchmark and real-world datasets verify the effectiveness of the proposed models compared with some state-of-the-art transductive learning models.

The remainder of this paper is organized as follows. Section 2 briefly introduces the background knowledge. Section 3 presents the proposed frameworks (TLFVS and TLFCV) and analyzes their risk bounds. Section 4 elaborates on the LSSVM-based $\text{TLFVS}_{\text{LSSVM}}$ and $\text{TLFCV}_{\text{LSSVM}}$. Section 5 arranges some preliminary experiments, and Section 6 concludes the paper.

2. Background

Consider a transductive learning problem with m samples $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m) \in \mathbb{R}^{d\times m}$ and their known and unknown labels $\mathbf{y} = (y_1, y_2, \ldots, y_m)^\top \in \{\pm 1\}^m$. Let $\mathbb{N}$, $\mathbb{N}_l$ and $\mathbb{N}_u$ denote the index sets of all samples, labeled samples, and unlabeled samples, respectively. Accordingly, $\mathbf{X}$ is partitioned into $\mathbf{X}_l \in \mathbb{R}^{d\times|\mathbb{N}_l|}$ and $\mathbf{X}_u \in \mathbb{R}^{d\times|\mathbb{N}_u|}$ with labels $\mathbf{y}_l$ and $\mathbf{y}_u$.

2.1. Transductive learning models

Transductive support vector machine (TSVM) (Joachims, 1999), the classical transductive learning model, seeks a decision hyperplane that separates the labeled samples while passing through low-density regions of the unlabeled data. It jointly optimizes the hyperplane with the normal vector $\mathbf{w} \in \mathbb{R}^d$ and bias $b \in \mathbb{R}$ and the unknown labels $\mathbf{y}_u$ by

$$\min_{\mathbf{w},b,\{y_i\}_{i\in\mathbb{N}_u}} \frac{1}{2}\|\mathbf{w}\|^2 + c\sum_{i\in\mathbb{N}_l} \ell\big(y_i f(\mathbf{x}_i)\big) + c^*\sum_{i\in\mathbb{N}_u} \ell\big(y_i f(\mathbf{x}_i)\big), \tag{1}$$

where $\ell(\cdot)$ is the hinge loss (Joachims, 1999), and parameters $c, c^* > 0$ control the trade-off between regularization and the losses on the labeled and unlabeled samples. Obviously, problem (1) minimizes the empirical risk, and it is a mixed integer programming problem, which is NP-hard.

Joachims (1999) proposed a local search heuristic that alternates between the real and integer parts. However, this strategy does not scale well to large datasets (Collobert et al., 2006). Chapelle and Zien (2005) showed that the unlabeled penalty in TSVM can be rewritten as a symmetric hinge loss. Thus, they proposed ∇TSVM, which replaces the nonsmooth loss with differentiable surrogates and uses a squared hinge loss for the labeled samples. This turns the original combinatorial problem into a continuous optimization problem that can be handled by gradient-based solvers. Alternatively, Collobert et al. (2006) addressed the nonconvexity using difference-of-convex programming. By introducing the ramp loss and assigning two possible labels to each unlabeled sample, they reformulated the objective in a difference-of-convex form. This allows the use of the concave-convex procedure (CCCP) (Collobert et al., 2006), which iteratively linearizes the concave part and solves a sequence of convex subproblems. As a result, the difference-of-convex strategy has become a common paradigm for developing robust TSVM variants (Cevikalp & Franc, 2017; Singla et al., 2021).

Another classical transductive model is transductive linear discrimination (TLD) (Chapelle et al., 1999). TLD regards a candidate labeling as proper if the ridge regression trained on labeled and unlabeled samples yields a small leave-one-out (LOO) error. It further adds a constraint, which keeps the final prediction close to the inductive solution learned on the labeled samples. The unknown labels are treated as continuous variables, and the objective becomes a convex quadratic function, so its solution follows by setting the gradient to zero, and the predicted labels are recovered by taking signs. However, the constraint above makes the result depend on the reliability of the prior information, and labels are relaxed from discrete to continuous. Although relaxation is often adopted for solving complex integer programming problems, solving them directly over the discrete feasible set avoids the loss caused by the relaxation.

2.2. Least squares support vector machine

LSSVM (Suykens & Vandewalle, 1999) reformulates the support vector machine (SVM) (Cortes & Vapnik, 1995) with a squared loss under equality constraints, seeking a decision function $f(\mathbf{x}) = \mathbf{w}^\top\mathbf{x} + b$. Given the training set $\mathbf{X}$ and $\mathbf{y}$, the optimization problem is

$$\begin{aligned} &\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + \frac{c}{2}\|\boldsymbol{\xi}\|^2 \\ &\text{s.t. } \phi(\mathbf{X})^\top\mathbf{w} + b\mathbf{e} = \mathbf{y} - \boldsymbol{\xi}, \end{aligned} \tag{2}$$

where $c > 0$ is a regularization parameter, $\boldsymbol{\xi} \in \mathbb{R}^m$ is a slack vector, $\phi(\cdot)$ maps the feature space into a reproducing kernel Hilbert space, and $\mathbf{e}$ is the all-one vector. Let $\boldsymbol{\alpha} \in \mathbb{R}^m$ be the Lagrange multiplier vector. The Karush-Kuhn-Tucker (KKT) (Suykens & Vandewalle, 1999) conditions are

$$\mathbf{w} = \phi(\mathbf{X})^\top\boldsymbol{\alpha},\ \mathbf{e}^\top\boldsymbol{\alpha} = 0,\ \boldsymbol{\xi} = \frac{1}{c}\boldsymbol{\alpha},\ \text{and } \phi(\mathbf{X})^\top\mathbf{w} + b\mathbf{e} = \mathbf{y} - \boldsymbol{\xi}. \tag{3}$$

After some algebra, the following equation holds:

$$\begin{pmatrix}\alpha\\ b\end{pmatrix}=\begin{pmatrix}\mathbf{K}(\mathbf{X},\mathbf{X})+\frac{1}{c}\mathbf{I} & \mathbf{e}\\ \mathbf{e}^{\top} & 0\end{pmatrix}^{-1}\begin{pmatrix}\mathbf{y}\\ 0\end{pmatrix}, \tag{4}$$

where $\mathbf{K}(\mathbf{X},\mathbf{X})$ is the kernel matrix constructed on the training set with its elements defined by a kernel function $k(\mathbf{x}_i,\mathbf{x}_j)=\phi(\mathbf{x}_i)^{\top}\phi(\mathbf{x}_j)$, and $\mathbf{I}$ is the identity matrix. Here $\mathbf{I}\in\mathbb{R}^{m\times m}$ and $\mathbf{e}\in\mathbb{R}^{m}$, so the coefficient matrix in (4) is of order $m+1$. Finally, given an unknown sample $\mathbf{x}$, the decision value is

$$f(\mathbf{x})=\begin{pmatrix}\mathbf{K}(\mathbf{x},\mathbf{X}) & 1\end{pmatrix}\begin{pmatrix}\mathbf{K}(\mathbf{X},\mathbf{X})+\frac{1}{c}\mathbf{I} & \mathbf{e}\\ \mathbf{e}^{\top} & 0\end{pmatrix}^{-1}\begin{pmatrix}\mathbf{y}\\ 0\end{pmatrix}, \tag{5}$$

and the predicted label is $\hat{y}=\operatorname{sign}(f(\mathbf{x}))$.

3. Validation risk minimization frameworks for transductive learning

Before presenting our transductive learning frameworks, we summarize the validation risk minimization principle of supervised learning as the following bilevel programming problem

$$\begin{aligned}&\min_{\theta}\quad \mathcal{E}\left(f^{*};\mathbf{X}_{\mathrm{val}}\right)\\ &\text{s.t.}\quad f^{*}=\arg\min_{f\in\mathcal{F}}\mathcal{J}\left(f;\mathbf{X}_{\mathrm{tr}};\theta\right),\end{aligned} \tag{6}$$

where $\mathcal{F}$ is a hypothesis space for the decision function f, $\mathcal{J}$ is a training objective, and $\mathcal{E}$ is a validation criterion. $\mathbf{X}_{\mathrm{val}}$ and $\mathbf{X}_{\mathrm{tr}}$ denote the validation and training sets, respectively. θ denotes the variables of learning machine f. Note that $\mathbf{X}_{\mathrm{tr}}$ and $\mathbf{X}_{\mathrm{val}}$ denote roles rather than fixed sets. $\mathbf{X}_{\mathrm{tr}}$ is the part of the data that takes part in the inner training, and $\mathbf{X}_{\mathrm{val}}$ is the part that evaluates f^{*}. Different ways of filling these two roles give different members of the framework. The above formulation is often used for hyperparameter selection (Liu et al., 2021; Pham et al., 2021; Shu et al., 2019), and we transform it into transductive learning to propose our frameworks.

3.1. Transductive learning framework by validation set (TLFVS)

Taking $\mathbf{X}_l$ as the validation set and $\mathbf{X}_u$ as the training set, our TLFVS becomes

$$\begin{aligned}&\min_{\{y_i\}_{i\in\mathbb{N}_u}}\ \sum_{i\in\mathbb{N}_l}\left|\operatorname{sign}(f^{*}(\mathbf{x}_i))-y_i\right|\\ &\quad\text{s.t. } f^{*}=\arg\min_{f}\mathcal{J}\left(f;\{(\mathbf{x}_i,y_i)\mid i\in\mathbb{N}_u\}\right),\\ &\qquad y_i\in\{-1,+1\},\ i\in\mathbb{N}_u.\end{aligned} \tag{7}$$

For each candidate assignment, a classifier f^{*} is trained on the unlabeled data at the inner level. At the outer level, the number of errors that this classifier makes on the labeled data is counted. The candidate with the minimal validation error is selected. By choosing a suitable learning machine f in (7) and solving it, the labels of unlabeled samples are recovered.

3.2. Transductive learning framework by cross-validation (TLFCV)

The quality of recovered labels by our TLFVS relies on the quality of the labeled samples. When the labeled samples are extremely scarce, our TLFVS may be unstable. Thus, we utilize the cross-validation criterion (Bengio & Grandvalet, 2004) to improve the robustness of our TLFVS. We split $\mathbb{N}_u$ into k folds $\{\mathbb{N}_{u_p}\}_{p=1}^{k}$. For each fold p, a learner f_p^{*} is trained on $\mathbb{N}_u\setminus\mathbb{N}_{u_p}$ and validated on $\mathbb{N}_{u_p}$. Our TLFCV is formulated as

$$\begin{aligned}&\min_{\{y_i\}_{i\in\mathbb{N}_u}}\ \lambda\sum_{i\in\mathbb{N}_l}\left|\operatorname{sign}(f^{*}(\mathbf{x}_i))-y_i\right|+\sum_{p=1}^{k}\sum_{i\in\mathbb{N}_{u_p}}\left|\operatorname{sign}(f_p^{*}(\mathbf{x}_i))-y_i\right|\\ &\quad\text{s.t. } f^{*}=\arg\min_{f}\mathcal{J}(f;\{(\mathbf{x}_i,y_i)\mid i\in\mathbb{N}_u\}),\\ &\qquad f_p^{*}=\arg\min_{f_p}\mathcal{J}(f_p;\{(\mathbf{x}_i,y_i)\mid i\in\mathbb{N}_u\setminus\mathbb{N}_{u_p}\}),p=1,\dots,k,\\ &\qquad y_i\in\{-1,+1\},i\in\mathbb{N}_u,\end{aligned} \tag{8}$$

where hyperparameter $\lambda>0$ controls the trade-off between the two objectives. Here, f^{*} is trained on all unlabeled data, while each f_p^{*} is trained on all folds except fold p. Minimizing this objective ensures that the label assignment fits the labeled data well while remaining stable across different subsets of the unlabeled data.

3.3. Discussion

This subsection compares our TLFVS and TLFCV theoretically. Both models search the candidate space $\mathcal{Y}_u=\{-1,+1\}^{|\mathbb{N}_u|}$. For a candidate $\tilde{\mathbf{y}}_u$, let $\hat{R}_l(\tilde{\mathbf{y}}_u)$ denote the validation error, that is, the outer objective of (7) scaled by $\frac{1}{2|\mathbb{N}_l|}$, and let $R(\tilde{\mathbf{y}}_u)$ denote the transductive error on $\mathbb{N}_u$. The two errors are defined on the same labeling of the whole sample. Both quantities lie in $[0,1]$.

Lemma 1 (El-Yaniv & Pechyony, 2009). *Suppose that $\mathbb{N}_l$ is drawn uniformly at random without replacement from $\mathbb{N}$. Then, for any fixed $\tilde{\mathbf{y}}_u\in\mathcal{Y}_u$ and any $\epsilon>0$,*

$$\mathbb{P}\left\{R(\tilde{\mathbf{y}}_u)-\hat{R}_l(\tilde{\mathbf{y}}_u)\geq\epsilon\right\}\leq\exp\left(-\frac{2\epsilon^2}{\tau q}\right), \tag{9}$$

where $\tau=\dfrac{|\mathbb{N}|}{(|\mathbb{N}|-\frac{1}{2})\left(1-\frac{1}{2\max(|\mathbb{N}_l|,|\mathbb{N}_u|)}\right)}$ is a finite-population correction factor and $q=\frac{1}{|\mathbb{N}_l|}+\frac{1}{|\mathbb{N}_u|}$.

Corollary 1. *In transduction the candidate space is finite, $|\mathcal{Y}_u|=2^{|\mathbb{N}_u|}$ (Vapnik, 1998). Hence for any $\mathcal{H}\subseteq\mathcal{Y}_u$ independent of $\mathbf{y}_l$, with probability at least $1-\delta$, every $\tilde{\mathbf{y}}_u\in\mathcal{H}$ satisfies*

$$R(\tilde{\mathbf{y}}_u)\leq\hat{R}_l(\tilde{\mathbf{y}}_u)+\sqrt{\frac{\ln|\mathcal{H}|+\ln(1/\delta)}{2}\left(\frac{1}{|\mathbb{N}_l|}+\frac{1}{|\mathbb{N}_u|}\right)}. \tag{10}$$

Proof. We require the conclusion to hold for all of $\mathcal{H}$ simultaneously with probability at least $1-\delta$, so each candidate is allowed a failure probability of $\delta/|\mathcal{H}|$. Setting the right side of Lemma 1 equal to $\delta/|\mathcal{H}|$ and solving for ϵ gives $\epsilon=\sqrt{\frac{\tau q}{2}(\ln|\mathcal{H}|+\ln(1/\delta))}$. Lemma 1 applies to each candidate in $\mathcal{H}$, since $\mathcal{H}$ does not depend on $\mathbf{y}_l$ and its candidates are fixed before $\mathbb{N}_l$ is drawn. A union bound over $\mathcal{H}$ then bounds the total failure probability by δ. Taking the complement and dropping τ, which affects none of the rates below, gives (10). □

The confidence term of (10) is thus the price of selecting a candidate instead of fixing one in advance. Here $\frac{1}{|\mathbb{N}_l|}+\frac{1}{|\mathbb{N}_u|}$ sets the noise scale and is dominated by the smaller of the two, while $|\mathcal{H}|$ enters only through $\ln|\mathcal{H}|$, so that only an exponential reduction of the candidate space tightens the bound appreciably.

Let $\mathcal{L}_{\mathrm{CV}}(\tilde{\mathbf{y}}_u)$ denote the k-fold cross-validation error, i.e., the second term of the TLFCV objective (8) scaled by $\frac{1}{2|\mathbb{N}_u|}$. Suppose the following two assumptions hold. The exponential decay in (A2) is motivated by Hoeffding's inequality.

(A1) TLFVS satisfies $\hat{R}_l(\tilde{\mathbf{y}}_u^{\mathrm{TLFVS}})\leq\epsilon_1$, and TLFCV satisfies $\lambda\,\hat{R}_l(\tilde{\mathbf{y}}_u^{\mathrm{TLFCV}})+\mathcal{L}_{\mathrm{CV}}(\tilde{\mathbf{y}}_u^{\mathrm{TLFCV}})\leq\epsilon_2$. The two validation errors are of the same order, i.e., $|\epsilon_1-\epsilon_2/\lambda|=O(1)$.

(A2) There is a constant $c_\theta>0$ such that, for the k-fold cross-validation loss and any $\epsilon_2<\frac{1}{2}$, the proportion of candidate labels in $\mathcal{Y}_u$ with $\mathcal{L}_{\mathrm{CV}}\leq\epsilon_2$ is bounded by $\exp\left(-c_\theta|\mathbb{N}_u|(\frac{1}{2}-\epsilon_2)^2\right)$.

Theorem 1. *Under assumptions (A1) and (A2), and assuming* $|\mathbb{N}_l|/|\mathbb{N}_u| = o(1)$*, with probability at least* $1-\delta$*,*

$$U_{TLFVS} - U_{TLFCV} = \Omega\left(\sqrt{\frac{|\mathbb{N}_u|}{|\mathbb{N}_l|}}\right), \tag{11}$$

where[1] U_{TLFVS} *and* U_{TLFCV} *are the transductive bounds of the TLFVS and TLFCV.*

Proof. We first establish the bounds for the TLFVS and TLFCV, then take the difference. TLFVS does not restrict the candidate space, so the cardinality of its effective hypothesis space is $\mathcal{N}_{\text{TLFVS}} = |\mathcal{Y}_u| = 2^{|\mathbb{N}_u|}$. With $\hat{R}_l(\tilde{\mathbf{y}}_u^{\text{TLFVS}}) \le \epsilon_1$, (10) gives $R(\tilde{\mathbf{y}}_u^{\text{TLFVS}}) \le U_{\text{TLFVS}}$, where

$$U_{\text{TLFVS}} = \epsilon_1 + \sqrt{\frac{|\mathbb{N}_u|\ln 2 + \ln(1/\delta)}{2}\left(\frac{1}{|\mathbb{N}_l|} + \frac{1}{|\mathbb{N}_u|}\right)}. \tag{12}$$

The cross-validation loss $\mathcal{L}_{\text{CV}}$ restricts the candidate labels to the subset $\mathcal{H}_{\text{CV}}(\epsilon_2) = \{\tilde{\mathbf{y}}_u \in \mathcal{Y}_u : \mathcal{L}_{\text{CV}}(\tilde{\mathbf{y}}_u) \le \epsilon_2\}$. Since $\mathcal{L}_{\text{CV}}$ is computed solely from $\mathbf{X}_u$ and the candidate labeling $\tilde{\mathbf{y}}_u$, it does not depend on $\mathbf{y}_l$, so $\mathcal{H}_{\text{CV}}(\epsilon_2)$ satisfies the hypothesis of Corollary 1. The candidate set of TLFVS is the whole $\mathcal{Y}_u$, which involves no data at all and therefore satisfies the hypothesis as well. Since $\mathcal{L}_{\text{CV}} \ge 0$, (A1) gives $\hat{R}_l(\tilde{\mathbf{y}}_u^{\text{TLFCV}}) \le \epsilon_2/\lambda$. Applying (10) over $\mathcal{H}_{\text{CV}}(\epsilon_2)$ with cardinality $\mathcal{N}_{\text{TLFCV}} = |\mathcal{H}_{\text{CV}}(\epsilon_2)|$ gives $R(\tilde{\mathbf{y}}_u^{\text{TLFCV}}) \le U_{\text{TLFCV}}$, where

$$U_{\text{TLFCV}} = \frac{\epsilon_2}{\lambda} + \sqrt{\frac{\ln \mathcal{N}_{\text{TLFCV}} + \ln(1/\delta)}{2}\left(\frac{1}{|\mathbb{N}_l|} + \frac{1}{|\mathbb{N}_u|}\right)}. \tag{13}$$

By (A2), $\mathcal{N}_{\text{TLFCV}}/\mathcal{N}_{\text{TLFVS}} \le \exp\big(-c_\theta|\mathbb{N}_u|(\frac{1}{2}-\epsilon_2)^2\big)$, hence $\ln\mathcal{N}_{\text{TLFVS}} - \ln\mathcal{N}_{\text{TLFCV}} \ge c_\theta|\mathbb{N}_u|(\frac{1}{2}-\epsilon_2)^2 = \Omega(|\mathbb{N}_u|)$. Denote the quantities under the roots by $A = \frac{1}{2}\big(\ln\mathcal{N}_{\text{TLFVS}} + \ln(1/\delta)\big)q$ and $B = \frac{1}{2}\big(\ln\mathcal{N}_{\text{TLFCV}} + \ln(1/\delta)\big)q$. Their difference satisfies $A - B = \frac{1}{2}\big(\ln\mathcal{N}_{\text{TLFVS}} - \ln\mathcal{N}_{\text{TLFCV}}\big)q \ge \frac{c_\theta}{2}(\frac{1}{2}-\epsilon_2)^2\frac{|\mathbb{N}_u|}{|\mathbb{N}_l|} = \Omega\Big(\frac{|\mathbb{N}_u|}{|\mathbb{N}_l|}\Big)$. Also $|\mathbb{N}_l| \le |\mathbb{N}_u|$ gives $q \le 2/|\mathbb{N}_l|$, hence $\sqrt{A} = O\big(\sqrt{|\mathbb{N}_u|/|\mathbb{N}_l|}\big)$. By $\sqrt{A}-\sqrt{B} \ge (A-B)/(2\sqrt{A})$, the difference of the two bounds is

$$\begin{aligned} U_{\text{TLFVS}} - U_{\text{TLFCV}} &= (\epsilon_1 - \epsilon_2/\lambda) + (\sqrt{A}-\sqrt{B}) \ge -O(1) + \frac{\Omega(|\mathbb{N}_u|/|\mathbb{N}_l|)}{O\big(\sqrt{|\mathbb{N}_u|/|\mathbb{N}_l|}\big)} \\ &= \Omega\left(\sqrt{\frac{|\mathbb{N}_u|}{|\mathbb{N}_l|}}\right). \end{aligned} \tag{14}$$

□

Remark 1. Theorem 1 demonstrates that the bound gap is positive and grows at least on the order of $\sqrt{|\mathbb{N}_u|/|\mathbb{N}_l|}$. This theoretically shows that TLFCV achieves a tighter transductive bound than TLFVS, and its advantages are more prominent when labeled samples are scarce ($|\mathbb{N}_l| \ll |\mathbb{N}_u|$).

4. Instantiations

This section instantiates our TLFVS and TLFCV using LSSVM as the learning machine to obtain our $\text{TLFVS}_{\text{LSSVM}}$ and $\text{TLFCV}_{\text{LSSVM}}$. LSSVM is chosen because its solution is available in closed form. Substituting it into (7) and (8) removes the inner problem and leaves a single-level integer program.

4.1. $TLFVS_{LSSVM}$

Before introducing our $\text{TLFVS}_{\text{LSSVM}}$, we handle the discontinuous objective of problem (7). Note that the following equivalence holds true:

$$\min \sum_{i\in\mathbb{N}_l}\Big|\text{sign}(f^*(\mathbf{x}_i)) - y_i\Big| \iff \max \sum_{i\in\mathbb{N}_l}\text{sign}\big(y_i f^*(\mathbf{x}_i)\big). \tag{15}$$

[1] $\Omega(\cdot)$ is the standard asymptotic lower bound: $g = \Omega(h)$ means that g is at least a constant multiple of h.

Relaxing the $\text{sign}(\cdot)$ operator linearly and substituting the LSSVM into problem (7), our $\text{TLFVS}_{\text{LSSVM}}$ is formulated as

$$\begin{aligned} \max_{\{y_i\}_{i\in\mathbb{N}_u}} & \sum_{i\in\mathbb{N}_l} y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) \\ \text{s.t.}\quad & (\mathbf{w}^*, b^*) = \arg\min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|_2^2 + \tfrac{c}{2}\sum_{i\in\mathbb{N}_u}\big(1 - y_i(\mathbf{w}^\top\mathbf{x}_i + b)\big)^2, \\ & y_i \in \{-1,+1\},\ i\in\mathbb{N}_u. \end{aligned} \tag{16}$$

The objective in (7) weights every labeled sample equally, whereas the relaxed objective weights each sample by its functional margin.

The inner problem of (16) trains an LSSVM on $\mathbf{X}_u$ with the candidate labels $\mathbf{y}_u$, so its solution is given by (5) with $\mathbf{X}_u$ in place of $\mathbf{X}$. Substituting this closed-form solution into the outer objective of bilevel problem (16) reduces it to

$$\begin{aligned} \max_{\mathbf{y}_u}\quad & \mathbf{y}_l^\top\big(\mathbf{K}(\mathbf{X}_l,\mathbf{X}_u) \quad \mathbf{e}\big)\begin{pmatrix}\mathbf{K}(\mathbf{X}_u,\mathbf{X}_u) + \frac{1}{c}\mathbf{I} & \mathbf{e}\\ \mathbf{e}^\top & 0\end{pmatrix}^{-1}\begin{pmatrix}\mathbf{y}_u\\ 0\end{pmatrix} \\ \text{s.t.}\quad & \mathbf{y}_u \in \{-1,+1\}^{|\mathbb{N}_u|}, \end{aligned} \tag{17}$$

which is a linear integer programming problem. Its global solution can be found easily. Let $\mathbf{z}$ be the solution of the linear system

$$\begin{pmatrix}\mathbf{K}(\mathbf{X}_u,\mathbf{X}_u) + \frac{1}{c}\mathbf{I} & \mathbf{e}\\ \mathbf{e}^\top & 0\end{pmatrix}\mathbf{z} = \begin{pmatrix}\mathbf{K}(\mathbf{X}_l,\mathbf{X}_u)^\top\mathbf{y}_l\\ \mathbf{e}^\top\mathbf{y}_l\end{pmatrix}. \tag{18}$$

By (18), problem (17) can be regarded as maximizing $\mathbf{z}^\top(\mathbf{y}_u;0)$. For each $y_i \in \mathbf{y}_u$, the optimal assignment has the same sign as its coefficient, i.e.,

$$y_i^* = \text{sign}(z_i), i = 1,2,\dots,|\mathbb{N}_u|. \tag{19}$$

The complete procedure of our $\text{TLFVS}_{\text{LSSVM}}$ is summarized in Algorithm 1.

Algorithm 1: $\text{TLFVS}_{\text{LSSVM}}$.

Input: Labeled data $(\mathbf{X}_l, \mathbf{y}_l)$, unlabeled data $\mathbf{X}_u$, regularization parameter c, kernel function $k(\cdot,\cdot)$.
Output: $\mathbf{y}_u$.
Solve the linear system (18) for $\mathbf{z}$;
for $i = 1,2,\dots,|\mathbb{N}_u|$ **do**
  $y_i \leftarrow \text{sign}(z_i)$;
end
return $\mathbf{y}_u$;

The main cost of Algorithm 1 is solving a $(|\mathbb{N}_u|+1)\times(|\mathbb{N}_u|+1)$ linear system, which has $\mathcal{O}(|\mathbb{N}_u|^3)$ time complexity.

4.2. $TLFCV_{LSSVM}$

Based on our $\text{TLFVS}_{\text{LSSVM}}$ and TLFCV, our $\text{TLFCV}_{\text{LSSVM}}$ is formulated as

$$\begin{aligned} \max_{\{y_i\}_{i\in\mathbb{N}_u}} & \lambda\sum_{i\in\mathbb{N}_l} y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) + \sum_{p=1}^{k}\sum_{i\in\mathbb{N}_{u_p}} y_i(\mathbf{w}_p^{*\top}\mathbf{x}_i + b_p^*) \\ \text{s.t.}\ & (\mathbf{w}^*, b^*) = \arg\min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|_2^2 + \tfrac{c}{2}\sum_{i\in\mathbb{N}_u}\big(1 - y_i(\mathbf{w}^\top\mathbf{x}_i + b)\big)^2, \\ & (\mathbf{w}_p{}^*, b_p^*) = \arg\min_{\mathbf{w}_p,b_p} \tfrac{1}{2}\|\mathbf{w}_p\|_2^2 + \tfrac{c}{2}\sum_{i\in\mathbb{N}_u\setminus\mathbb{N}_{u_p}}\big(1 - y_i(\mathbf{w}_p^\top\mathbf{x}_i + b_p)\big)^2, \\ & p = 1,2,\dots,k, \\ & y_i \in \{-1,+1\},\ i\in\mathbb{N}_u, \end{aligned} \tag{20}$$

where $\mathbf{w}_p$ and b_p are the normal vector and bias of the hyperplane in the pth fold, respectively. The first term in problem (20) is the same as in the $\text{TLFVS}_{\text{LSSVM}}$ (16) and the second term is the relaxation of the cross-validation criterion. Substituting the LSSVM's closed-form solution (5)

into problem (20), we obtain

$$\begin{aligned}\max_{\mathbf{y}_u}\ & \lambda \mathbf{y}_l^\top \begin{pmatrix}\mathbf{K}(\mathbf{X}_l,\mathbf{X}_u) & \mathbf{e}\end{pmatrix}\begin{pmatrix}\mathbf{K}(\mathbf{X}_u,\mathbf{X}_u)+\frac{1}{c}\mathbf{I} & \mathbf{e}\\ \mathbf{e}^\top & 0\end{pmatrix}^{-1}\begin{pmatrix}\mathbf{y}_u\\ 0\end{pmatrix}\\ &+\sum_{p=1}^{k}\mathbf{y}_{u_p}^\top\begin{pmatrix}\mathbf{K}(\mathbf{X}_{u_p},\bar{\mathbf{X}}_{u_p}) & \mathbf{e}\end{pmatrix}\begin{pmatrix}\mathbf{K}(\bar{\mathbf{X}}_{u_p},\bar{\mathbf{X}}_{u_p})+\frac{1}{c}\mathbf{I} & \mathbf{e}\\ \mathbf{e}^\top & 0\end{pmatrix}^{-1}\begin{pmatrix}\bar{\mathbf{y}}_{u_p}\\ 0\end{pmatrix}\\ \text{s.t. }& \mathbf{y}_u\in\{\pm1\}^{|\mathbb{N}_u|},\end{aligned}\tag{21}$$

where $\mathbf{X}_{u_p}$ denotes the samples for the fold p in $\mathbf{X}_u$, $\bar{\mathbf{X}}_{u_p}=\mathbf{X}_u\setminus\mathbf{X}_{u_p}$, $\bar{\mathbf{y}}_{u_p}=\mathbf{y}_u\setminus\mathbf{y}_{u_p}$.

After some algebra, problem (21) can be rewritten as

$$\begin{aligned}\max_{\mathbf{y}_u}\ & \mathbf{y}_u^\top\mathbf{H}\mathbf{y}_u+\mathbf{g}^\top\mathbf{y}_u\\ \text{s.t. }& \mathbf{y}_u\in\{-1,+1\}^{|\mathbb{N}_u|},\end{aligned}\tag{22}$$

where $\mathbf{g}\in\mathbb{R}^{|\mathbb{N}_u|}$, $\mathbf{H}\in\mathbb{R}^{|\mathbb{N}_u|\times|\mathbb{N}_u|}$. Specifically,

$$\mathbf{g}=\lambda\begin{pmatrix}\mathbf{I}_{|\mathbb{N}_u|} & \mathbf{0}\end{pmatrix}\begin{pmatrix}\mathbf{K}(\mathbf{X}_u,\mathbf{X}_u)+\frac{1}{c}\mathbf{I} & \mathbf{e}\\ \mathbf{e}^\top & 0\end{pmatrix}^{-1}\begin{pmatrix}\mathbf{K}(\mathbf{X}_l,\mathbf{X}_u)^\top\\ \mathbf{e}^\top\end{pmatrix}\mathbf{y}_l\tag{23}$$

and

$$\mathbf{H}=\begin{pmatrix}\mathbf{0} & \mathbf{H}_{12} & \cdots & \mathbf{H}_{1k}\\ \mathbf{H}_{12}^\top & \mathbf{0} & \cdots & \mathbf{H}_{2k}\\ \vdots & & \ddots & \vdots\\ \mathbf{H}_{1k}^\top & \mathbf{H}_{2k}^\top & \cdots & \mathbf{0}\end{pmatrix},\tag{24}$$

where

$$\begin{aligned}\mathbf{H}_{pq}=\frac{1}{2}\Bigg(&\left[\begin{pmatrix}\mathbf{K}(\mathbf{X}_{u_p},\bar{\mathbf{X}}_{u_p}) & \mathbf{e}\end{pmatrix}\begin{pmatrix}\mathbf{K}(\bar{\mathbf{X}}_{u_p},\bar{\mathbf{X}}_{u_p})+\frac{1}{c}\mathbf{I} & \mathbf{e}\\ \mathbf{e}^\top & 0\end{pmatrix}^{-1}\right]_{\mathbf{X}_{u_q}}\\ &+\left[\begin{pmatrix}\mathbf{K}(\mathbf{X}_{u_q},\bar{\mathbf{X}}_{u_q}) & \mathbf{e}\end{pmatrix}\begin{pmatrix}\mathbf{K}(\bar{\mathbf{X}}_{u_q},\bar{\mathbf{X}}_{u_q})+\frac{1}{c}\mathbf{I} & \mathbf{e}\\ \mathbf{e}^\top & 0\end{pmatrix}^{-1}\right]^\top_{\mathbf{X}_{u_p}}\Bigg),\end{aligned}\tag{25}$$

and $[\,\cdot\,]_{\mathbf{X}_{u_q}}$ selects the columns of the matrix indexed by the samples $\mathbf{X}_{u_q}$.

Problem (22) is a quadratic integer programming problem, which is generally NP-hard (Kochenberger et al., 2014). It can be reformulated as either box-constrained concave programming (Raghavachari, 1969) or mixed integer linear programming (Adams & Sherali, 1986), but both approaches remain intractable for practical problem sizes. Converting it to linear integer programming (Watters, 1967) is theoretically possible but impractical. For example if $|\mathbb{N}_u|=1{,}000$, the reformulated linear integer programming problem would introduce 10^6 variables and 2×10^6 constraints. A closed-form solution exists only when $\mathbf{H}+\mathrm{diag}(\mathbf{g})$ is strictly diagonally dominant, which rarely holds in practice. Therefore, in the following we design a greedy algorithm to obtain as good a solution to problem (22) as possible.

Expanding problem (22) into scalar form gives

$$\begin{aligned}\max_{y_1,\ldots,y_{|\mathbb{N}_u|}}\ & \sum_{i=1}^{|\mathbb{N}_u|}\sum_{j=i+1}^{|\mathbb{N}_u|}2h_{ij}y_iy_j+\sum_{i=1}^{|\mathbb{N}_u|}g_iy_i\\ \text{s.t. }& y_i\in\{-1,+1\},\end{aligned}\tag{26}$$

where h_{ij} and g_i are elements of $\mathbf{H}$ and $\mathbf{g}$. Each term $h_{ij}y_iy_j$ encourages y_i and y_j to have the same sign if $h_{ij}>0$, or opposite signs if $h_{ij}<0$. Each linear term g_iy_i pushes y_i toward $\mathrm{sign}(g_i)$.

The greedy algorithm processes the coefficients in descending order of absolute value. It introduces two concepts, i.e., connection and bag. A connection is a pairwise relation between two variables, defined by h_{ij}. A bag is a group of variables that are linked through connections. When a positive connection $h_{ij}>0$ is processed, y_i and y_j are encouraged to share the same sign. When a negative connection $h_{ij}<0$ is processed, they are encouraged to take opposite signs. As connections accumulate, variables merge into larger bags. Each bag maintains a cumulative value that reflects the total contribution from its internal connections. When a linear term g_i is processed, it fixes the sign of y_i, and this decision propagates through the bag. If a bag's cumulative value becomes negative, the accepted connections are outweighed by the conflicting ones, so the algorithm discards those accepted connections, deletes the bag, and restarts from the earliest conflicting coefficient. The discarded connections are not processed again. Fig. 2 illustrates the mechanisms of connection and bag, as well as how the label assignment is performed.

For example, $h_{12}=5>0$ establishes a positive connection between y_1 and y_2, creating a bag with value 5. At $t=2$, $h_{34}=-3<0$ establishes a negative connection between y_3 and y_4, yielding a bag with value 3. At $t=3$, $h_{13}=2>0$ connects y_1 and y_3. Since y_1 and y_2 share a positive connection, and y_3 and y_4 share a negative connection, all four variables merge into a single bag with cumulative value $5+3+2=10$. Finally, at $t=4$, $g_1>0$ sets $y_1=+1$, which propagates through the bag to yield $y_2=+1$, $y_3=+1$, and $y_4=-1$. The complete procedure is given in Algorithm 2.

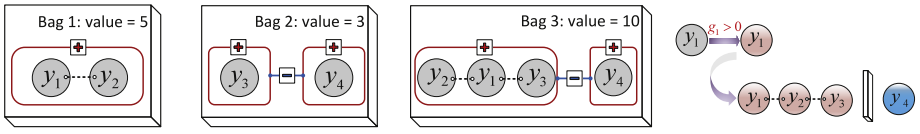
Algorithm 2: Greedy algorithm for quadratic integer programming problem (26).

Input: Coefficients $\{h_{ij}\}_{1\le i<j\le|\mathbb{N}_u|}$ and $\{g_i\}_{i=1}^{|\mathbb{N}_u|}$
Output: Labels $\mathbf{y}_u=\{y_1,\ldots,y_{|\mathbb{N}_u|}\}\in\{-1,+1\}^{|\mathbb{N}_u|}$

Build list $\mathbf{s}=[\{h_{ij}\},\{g_i\}]$ and sort it by $|\mathbf{s}|$ in descending order;
Initialize all y_i as unassigned, $\mathcal{B}\leftarrow\emptyset$, $t\leftarrow1$;
while $t\le|\mathbf{s}|$ **do**
  if $s_t\in\{h_{ij}\}$ and both y_i,y_j are unassigned **then**
    if neither y_i nor y_j is in a bag **then**
      Create a bag $B=y_i,y_j$ linked by $|s_t|$, with $v_B\leftarrow|s_t|$;
    else if $y_i\in B_1$ and $y_j\in B_2$ with $B_1\neq B_2$ **then**
      Merge bags and update $v_B\leftarrow v_{B_1}+v_{B_2}+|s_t|$;
    else if exactly one of y_i,y_j is in a bag B **then**
      Add the other one to B through s_t, and $v_B\leftarrow v_B+|s_t|$;
    else if y_i and y_j are in the same bag B **then**
      if s_t agrees with the connections in B **then**
        $v_B\leftarrow v_B+|s_t|$;
      else
        $v_B\leftarrow v_B-|s_t|$, and $t'\leftarrow t$ at the first decrease of v_B;
      end
      if $v_B<0$ **then**
        Discard the connections of B before t', delete B, and $t\leftarrow t'-1$
      end
      ;
    end
  else if $s_t\in\{h_{ij}\}$ and exactly one of y_i,y_j is assigned **then**
    Assign the other variable accordingly, then propagate;
  else if $s_t\in\{g_i\}$ and y_i is unassigned **then**
    $y_i\leftarrow\mathrm{sign}(s_t)$, then propagate through its bag;
  $t\leftarrow t+1$;
end
foreach remaining bag B **do**
  Assign all variables in B by its connections;
end
return $\mathbf{y}_u$;

The main cost of Algorithm 2 is sorting $\mathbf{s}$. With $|\mathbb{N}_u|$ unlabeled samples evenly split into $k\ge2$ folds, the list size is $\frac{k-1}{2k}|\mathbb{N}_u|^2+|\mathbb{N}_u|$. Thus, the time complexity is $\mathcal{O}\big(\frac{k-1}{k}|\mathbb{N}_u|^2\log|\mathbb{N}_u|\big)$. The following two theorems characterize the greedy algorithm.

Theorem 2. *Algorithm 2 terminates in a finite number of steps.*



(a) $t = 1, h_{12} = 5$ (b) $t = 2, h_{34} = -3$ (c) $t = 3, h_{13} = 2$ (d) $t = 4, g_1 > 0$

Fig. 2. Illustration of the greedy bag formation process. Samples are dynamically grouped through connections based on coefficient values.

Proof. Let $\mathbf{s}$ be the sorted coefficient list, which has finite length. The pointer t advances through $\mathbf{s}$ except when a contradiction triggers backtracking. Each backtracking event removes at least one conflicting connection from the active set, strictly reducing its size. Since the active set is finite, the number of backtracking events is bounded. Therefore, t eventually reaches the end of $\mathbf{s}$, and the algorithm terminates. □

Theorem 3. *Suppose* $\mathbf{H} + \mathrm{diag}(\mathbf{g})$ *is strictly diagonally dominant, i.e.,*

$$|g_i| > \sum_{j \neq i} |2h_{ij}|, \forall i = 1, 2, \ldots, |\mathbb{N}_u|.$$

Then Algorithm 2 returns the unique global solution to problem (22).

Proof. Under strict diagonal dominance, the contribution of each linear term $g_i y_i$ dominates the combined contribution of all quadratic terms involving y_i. Consequently, the unique maximizer of (22) is $y_i^* = \mathrm{sign}(g_i)$ for all i. Moreover, the conditions $|g_i| > |2h_{ij}|$ and $|g_j| > |2h_{ij}|$ imply that every linear term g_i appears before its associated quadratic terms h_{ij} in the sorted list $\mathbf{s}$. Hence the algorithm assigns $y_i = \mathrm{sign}(g_i)$ directly, which coincides with the global optimum. □

Since $\mathbf{g}$ scales linearly with λ by (23), the smallest λ that makes $\mathbf{H} + \mathrm{diag}(\mathbf{g})$ strictly diagonally dominant can be computed directly. Above this threshold the labeled samples dominate the outer objective, so λ should be kept below it for the cross-validation term to remain effective.

5. Experiments

In this section, we compare our TLFVS$_{\text{LSSVM}}$ and TLFCV$_{\text{LSSVM}}$ with several transductive learning models, including TLD (Chapelle et al., 1999), TSVM^{CCCP} (Collobert et al., 2006), ∇TSVM (Chapelle & Zien, 2005), RTSVM (Cevikalp & Franc, 2017), $\overline{\text{pin}}$-TSVM (Singla et al., 2021), and DIFFormer (Wu et al., 2023). DIFFormer is a deep transductive model built on the Transformer architecture, and it represents the recent deep learning approaches to transductive learning. We use its DIFFormer-s version. LSSVM (Suykens & Vandewalle, 1999) and ResNet-18 (He et al., 2016) serve as supervised baselines. For the benchmark and real-world datasets, we draw 20 random splits of each dataset, taking 20 labeled samples in proportion to the class sizes, and all the compared methods are evaluated on the same splits. The deep learning models are implemented in Python, and all the other methods are implemented in MATLAB. All experiments were run on a desktop workstation with an Intel Core i7-14700 CPU, an NVIDIA RTX 4070 SUPER GPU, and 32 GB of RAM. Our code and these data files are available.[2]

We use the Radial Basis Function (RBF) kernel $k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ (Chapelle & Zien, 2005) for all nonlinear experiments. For datasets with more than 1,000 samples, γ is set via the ideal kernel (Akram-Ali-Hammouri et al., 2021). On each dataset, all methods share the same γ. We define two search grids $\Pi_1 = \{2^{-8}, 2^{-7}, \ldots, 2^7\}$ and $\Pi_2 = \{2^{-10}, 2^{-9}, \ldots, 2^5\}$. Table 1 summarizes the hyperparameter ranges. For the proposed TLFCV$_{\text{LSSVM}}$, the cross-validation parameter k is taken as 5 unless stated otherwise. For DIFFormer-s, the learning rate, weight decay and dropout rate are searched over the ranges given in its paper, while the architecture is fixed at two layers with a hidden width of 64.

We evaluate the models under label scarcity to reflect practical transductive applications. In this setting, splitting the already scarce labeled samples for model selection is impractical, and cross-validation on such a small set tends to overfit the labeled set. Therefore, to measure the best performance of each model, we report its highest transductive accuracy. The same protocol is applied to each method.

Table 1
Hyperparameter search ranges

Method	Parameters	
LSSVM	$c \in \Pi_1$,	$\gamma \in \Pi_2$
TLD	$c^*, c \in \Pi_1$,	$\gamma \in \Pi_2$
TSVM^{CCCP}	$c^*, c \in \Pi_1$,	$\gamma \in \Pi_2$
∇TSVM	$c \in \Pi_1$,	$\gamma \in \Pi_2$
RTSVM	$c^*, c \in \Pi_1$,	$\gamma \in \Pi_2$
$\overline{\text{pin}}$-TSVM	$c^*, c \in \Pi_1$,	$\gamma \in \Pi_2$
TLFVS$_{\text{LSSVM}}$	$c \in \Pi_1$,	$\gamma \in \Pi_2$
TLFCV$_{\text{LSSVM}}$	$\lambda, c \in \Pi_1$,	$\gamma \in \Pi_2$

5.1. Synthetic datasets

1) *Two-Dumbbells*. In Fig. 3a, each dumbbell represents a class. With the linear kernel $k(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$ (Chapelle & Zien, 2005) and all parameters set to 1, Fig. 3 presents the results of evaluated methods. Both TSVM^{CCCP} and $\overline{\text{pin}}$-TSVM incorrectly cut through the sparse bridge. Although TLFVS$_{\text{LSSVM}}$ correctly classifies all the labeled samples, it still performs poorly on the unlabeled samples. TLFCV$_{\text{LSSVM}}$ recovers the underlying manifold structure for both $k = 2$ and $k = 5$.

These results show that the low-density assumption in TSVMs can be misleading when the labeled samples are scarce. TSVMs treat the sparse bridge as the optimal separation margin. Instead, our models use the labeled samples for validation, which gives the correct labels on the unlabeled samples. The gap between TLFVS$_{\text{LSSVM}}$ and TLFCV$_{\text{LSSVM}}$ also shows that the cross-validation on the unlabeled samples is effective.

2) *Two-Moons*. Two-Moons is a classical nonlinear dataset, in which the two classes are two interleaving moons. We use the RBF kernel with $\gamma = 1$, and the results are shown in Fig. 4.

TSVM^{CCCP} produces a localized decision region. Although $\overline{\text{pin}}$-TSVM performs better, it still misclassifies the distant tails of the moons. The same happens to TLFVS$_{\text{LSSVM}}$, which labels the distant samples wrongly. TLFCV$_{\text{LSSVM}}$ propagates the labels correctly across the entire structure. We attribute the failure of TSVM^{CCCP} to the kernel parameter. TLFVS$_{\text{LSSVM}}$ fails for another reason. Its outer objective only asks the labeled samples to be classified correctly, which is a weak restriction on the unlabeled samples when the labeled samples are few. TLFCV$_{\text{LSSVM}}$ raises the requirement on the candidate labelings through the cross-validation on the unlabeled samples, and therefore achieves a better result.

3) *Two-Magnets*. The data in Fig. 5a consist of two magnets, each split into a positive and a negative block, so the four blocks alternate in class and the third block carries no labeled sample. We compare $\overline{\text{pin}}$-TSVM with our TLFCV$_{\text{LSSVM}}$ on this dataset, and their results are shown in Fig. 5. With all parameters set to 1, the decision boundary of $\overline{\text{pin}}$-TSVM is determined by the two adjacent labeled samples from different classes. Tuning its parameters ($c = 0.1, c^* = 10$) then drives the decision boundary toward the low-density region, as shown in Fig. 5c. TLFCV$_{\text{LSSVM}}$ ($k = 5$) recovers the labels well on both magnets.

[2] https://github.com/longfollow233/TLFcode

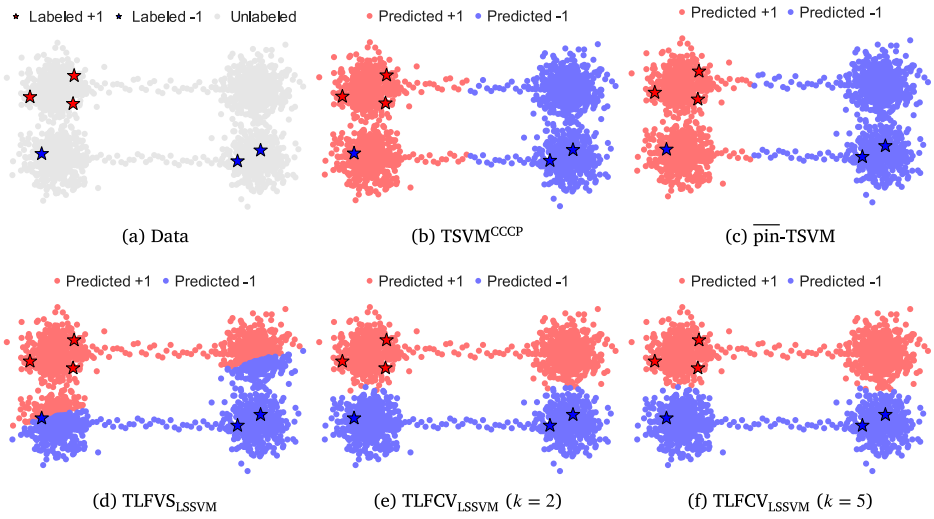


Fig. 3. Results on the Two-Dumbbells dataset. (a) The data, with three labeled samples per class. (b)–(f) The labels predicted by the compared methods.

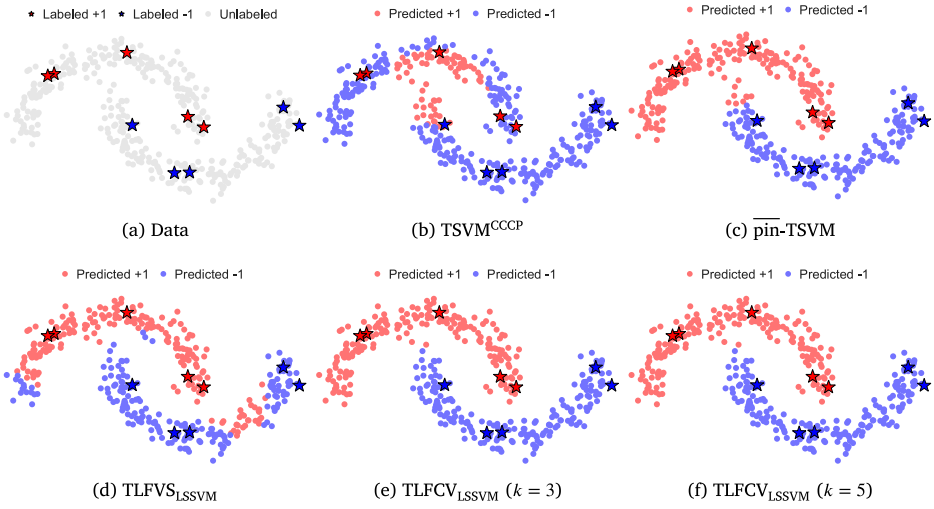


Fig. 4. Results on the Two-Moons dataset. (a) The data, with five labeled samples per class. (b)–(f) The labels predicted by the compared methods.

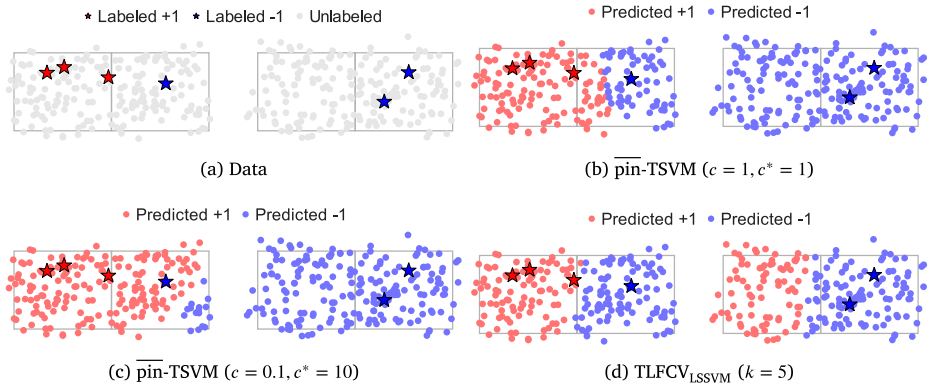
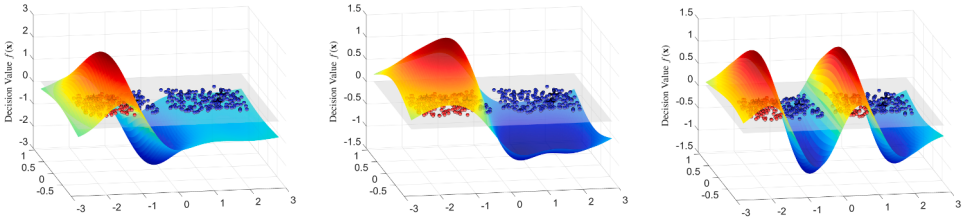


Fig. 5. Results on the Two-Magnets dataset. (a) The data, with three labeled samples per class. The two magnet-shaped boxes mark the range of the data distribution; within each magnet the left half is the positive class and the right half is the negative class. (b)–(d) The labels predicted by the compared methods.

We take the labels each method predicts, train a proxy model with these labels and plot its decision value $f(\mathbf{x})$ as shown in Fig. 6. The proxy is an SVM for $\overline{\text{pin}}$-TSVM and an LSSVM for TLFCV$_{\text{LSSVM}}$. The surface of $\overline{\text{pin}}$-TSVM crosses the zero level only once, both with the parameters in Fig. 6a and b. Tuning moves the crossing into the gap between the second and the third block. The surface of TLFCV$_{\text{LSSVM}}$ in Fig. 6c rises and falls across the four blocks. This comes from two sources. The KKT condition (3) ($\mathbf{e}^\top \boldsymbol{\alpha} = 0$) drives the inner LSSVM towards a balanced assignment of the two classes, while the cross-validation term filters out the candidate labelings that do not hold on every fold. Together they let TLFCV$_{\text{LSSVM}}$ adapt to the complex geometry of the Two-Magnets.



(a) $\overline{\text{pin}}$-TSVM ($c = 1, c^* = 1$) (b) $\overline{\text{pin}}$-TSVM ($c = 0.1, c^* = 10$) (c) $\text{TLFCV}_{\text{LSSVM}}$ ($k = 5$)

Fig. 6. Decision surfaces on the Two-Magnets dataset, obtained by training a proxy model on the labels each method predicts. The two horizontal axes are the coordinates of the data plane and the vertical axis is the decision value of the proxy model, which is also visualized by a color scale ranging from red for high values to blue for low ones.

Table 2
Details of the benchmark datasets.

Dataset	Name	Instances	Features	Classes
D_1	Hill-Valley	606	100	2
D_2	Raisin	900	7	2
D_3	Tictactoe	958	27	2
D_4	Diabetic	1,151	19	2
D_5	Titanic	2,201	3	2
D_6	Chess2	3,196	36	2
D_7	HIV	3,272	8	2
D_8	Wilt	4,839	5	2
D_9	Tic	5,822	85	2
D_{10}	Magic	19,020	10	2

5.2. Benchmark datasets

We evaluate the proposed models and all the compared methods on the benchmark datasets from the UCI Machine Learning Repository.[3] The details of these datasets are summarized in Table 2.

Table 3 reports the performance of all evaluated methods across the benchmark datasets. The boldface marks the best accuracy on each dataset, and the best average rank. $\text{TLFCV}_{\text{LSSVM}}$ has the best average rank 2.65, followed by DIFFormer with 3.00 and $\text{TLFVS}_{\text{LSSVM}}$ with 4.00. Following Demšar (2006), the Friedman test with the Iman-Davenport correction gives $F(8, 72) = 5.52$ with $p = 1.9 \times 10^{-5}$, so the nine methods do differ in performance. Holm's post-hoc test then takes $\text{TLFCV}_{\text{LSSVM}}$ as the control, and the adjusted p-values are given in the last row of Table 3. At the level 0.05, $\text{TLFCV}_{\text{LSSVM}}$ is significantly better than $\text{TSVM}^{\text{CCCP}}$, ∇TSVM, $\overline{\text{pin}}$-TSVM and LSSVM. Its advantage over RTSVM, TLD, $\text{TLFVS}_{\text{LSSVM}}$ and DIFFormer does not reach significance, since the test needs more datasets to separate close methods.

Two things can still be read from the table. First, $\text{TLFCV}_{\text{LSSVM}}$ is among the top three on seven of the ten datasets, which no other method reaches. Second, its worst rank over the ten datasets is the fifth, while DIFFormer, the closest competitor in average rank, falls to the eighth on D_3. $\text{TLFCV}_{\text{LSSVM}}$ performs well on most of the datasets, but it does not dominate the others. It is only one instantiation of the proposed frameworks, and more effective models can be built by changing the inner learner or the validation criterion.

5.3. Real-world dataset

We further evaluate the performance on the real-world PatchCamelyon (PCam) tumor dataset,[4] in which each image is a patch of a lymph node section and has to be labeled as tumor or normal. Fig. 7 shows images from the dataset, including tumor and normal instances.

A subset of 2,000 images is used in this experiment. Features are extracted using a pre-trained ResNet-18 (He et al., 2016) model and then fed into the evaluated transductive algorithms, while a fine-tuned ResNet-18 serves as the baseline. This experiment therefore differs from the benchmark ones in two ways. The inputs are deep features rather than the original attributes, and a supervised deep model is available for comparison. Owing to its prohibitive computational cost on this dataset, $\overline{\text{pin}}$-TSVM is excluded from this experiment. On this dataset we focus on the comparison with the TSVM family, which shares the same kernel-based setting as our models.

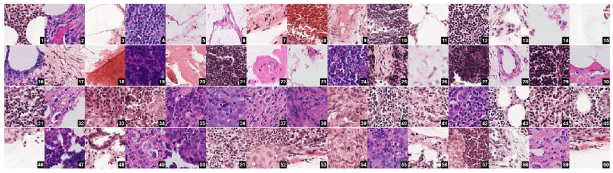


Fig. 7. Sample images from the PCam dataset; images 1–30 are normal and 31–60 are tumor.

Table 4 summarizes the mean accuracy, standard deviation, and average running time over the 20 trials. The boldface marks the best accuracy and the shortest runtime. The reported time corresponds to the classification stage on the shared pre-extracted features and excludes the feature-extraction cost, whereas the ResNet-18 time covers its complete fine-tuning and inference pipeline.

$\text{TLFCV}_{\text{LSSVM}}$ attains the highest accuracy (78.18%), and its standard deviation is the smallest among the transductive models. $\text{TLFVS}_{\text{LSSVM}}$ likewise surpasses the fine-tuned ResNet-18 baseline. The efficiency advantage is equally pronounced, $\text{TLFVS}_{\text{LSSVM}}$ classifies in about 1 second by solving a single linear system, and $\text{TLFCV}_{\text{LSSVM}}$ reaches the best accuracy in 307 seconds, one to two orders of magnitude faster than the $\text{TSVM}^{\text{CCCP}}$ and RTSVM. These results show that the proposed frameworks also work when the inputs are deep features, and that they reach a better accuracy than the supervised network from which the features come.

5.4. Further analysis

This subsection examines four questions left open by the comparison above, namely how the kernel parameter is selected, how stable the optimal hyperparameters are, how the cross-validation parameter k acts inside $\text{TLFCV}_{\text{LSSVM}}$, and how much the models cost to run.

1) *Kernel Parameter Selection.* For datasets with more than 1,000 samples the kernel parameter is set by the ideal kernel instead of a grid search, and this part examines whether the two give comparable results. Specifically, the ideal kernel parameter is selected by minimizing the mean absolute difference between the empirical kernel matrix and an ideal kernel matrix, which equals 1 for sample pairs of the same class and 0 otherwise (Akram-Ali-Hammouri et al., 2021). Since only the labeled samples carry class information in this setting, the kernel parameters are computed based on them. Denoting by $\mathbf{J}$ the ideal kernel matrix on

[3] https://archive.ics.uci.edu/
[4] https://github.com/basveeling/pcam

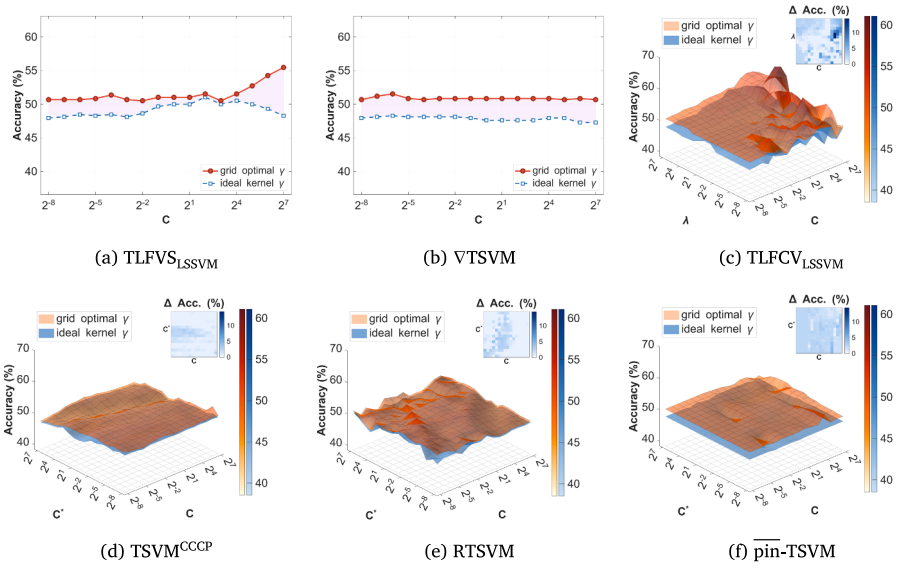
Table 3
Mean accuracy (%) and standard deviation over 20 trials on the benchmark datasets.

Dataset	LSSVM	TSVM^{CCCP}	VTSVM	RTSVM	$\overline{\text{pin}}$-TSVM	TLD	DIFFormer	TLFVS$_{LSSVM}$	TLFCV$_{LSSVM}$
D_1	52.33 ± 0.93	52.27 ± 0.57	51.86 ± 0.59	53.67 ± 0.45	53.08 ± 0.85	53.18 ± 1.03	56.42 ± 3.48	56.08 ± 2.10	**64.04 ± 2.19**
D_2	84.14 ± 2.54	74.49 ± 3.49	86.89 ± 0.47	**87.12 ± 0.34**	87.05 ± 0.36	85.46 ± 1.79	85.64 ± 1.65	84.42 ± 1.97	86.83 ± 0.38
D_3	68.36 ± 1.36	65.87 ± 0.31	68.00 ± 2.06	68.78 ± 1.53	71.26 ± 3.71	71.79 ± 3.08	67.73 ± 1.92	75.06 ± 4.25	**77.53 ± 4.53**
D_4	55.89 ± 3.35	53.11 ± 0.09	55.73 ± 3.48	56.19 ± 3.27	56.71 ± 3.78	57.71 ± 3.29	**61.09 ± 2.56**	56.41 ± 3.36	57.57 ± 3.99
D_5	75.93 ± 3.43	57.02 ± 10.37	74.91 ± 4.27	75.75 ± 4.39	77.63 ± 0.42	76.48 ± 3.08	78.32 ± 0.55	77.75 ± 1.01	**78.46 ± 0.49**
D_6	65.11 ± 4.69	58.04 ± 3.43	56.93 ± 5.62	64.63 ± 6.23	64.99 ± 6.33	70.14 ± 3.97	**75.39 ± 3.33**	65.64 ± 7.05	66.97 ± 5.89
D_7	86.75 ± 0.00	**86.76 ± 0.03**	86.75 ± 0.00	86.75 ± 0.00	72.88 ± 4.61	86.75 ± 0.01	86.75 ± 0.00	86.75 ± 0.00	86.75 ± 0.00
D_8	94.60 ± 0.00	94.60 ± 0.00	94.60 ± 0.00	94.60 ± 0.00	92.46 ± 3.91	94.60 ± 0.00	**95.09 ± 0.75**	94.62 ± 0.09	94.62 ± 0.07
D_9	94.02 ± 0.00	74.46 ± 21.86	94.02 ± 0.00	94.02 ± 0.01	93.49 ± 0.57	94.02 ± 0.00	94.02 ± 0.00	94.02 ± 0.01	94.02 ± 0.00
D_{10}	72.60 ± 2.99	OOM	69.42 ± 4.99	73.16 ± 3.71	OOM	72.84 ± 2.38	**75.04 ± 2.22**	72.84 ± 2.46	73.28 ± 2.79
Rank	6.00	7.65	6.60	4.80	6.05	4.25	3.00	4.00	**2.65**
Holm p	0.033	<0.001	0.009	0.317	0.033	0.574	0.775	0.574	–

Note: "OOM" denotes out of memory. Tied best values are not marked in boldface. "–" denotes the control method of Holm's post-hoc test, against which all other methods are compared.

Table 4
Mean accuracy (%) and average runtime (sec.) over 20 trials on the PCam dataset.

Data	Metric	ResNet-18	TSVM^{CCCP}	VTSVM	RTSVM	TLFVS$_{LSSVM}$	TLFCV$_{LSSVM}$
PCam	Accuracy	74.57 ± 4.15	61.56 ± 4.26	75.01 ± 9.27	77.47 ± 6.00	75.97 ± 5.74	**78.18 ± 4.15**
	Time	5.00	9131.00	523.00	42169.00	**1.00**	307.00



(a) TLFVS$_{LSSVM}$ (b) VTSVM (c) TLFCV$_{LSSVM}$

(d) TSVM^{CCCP} (e) RTSVM (f) $\overline{\text{pin}}$-TSVM

Fig. 8. Comparison of the ideal kernel strategy and exhaustive grid search on D_1. For each setting of a method's remaining hyperparameters, the orange curve or surface gives the accuracy under the best γ over the grid $\{2^{-10} : 2^5\}$ and the blue one the accuracy under the γ given by the ideal kernel criterion. (a)–(b) Line plots over the c, with the gap shaded between the curves. (c)–(f) Surfaces over the two hyperparameters, with the gap given by the inset heat map Δ Acc. (%). The vertical axis is the accuracy throughout.

the labeled samples, the kernel parameter is selected as

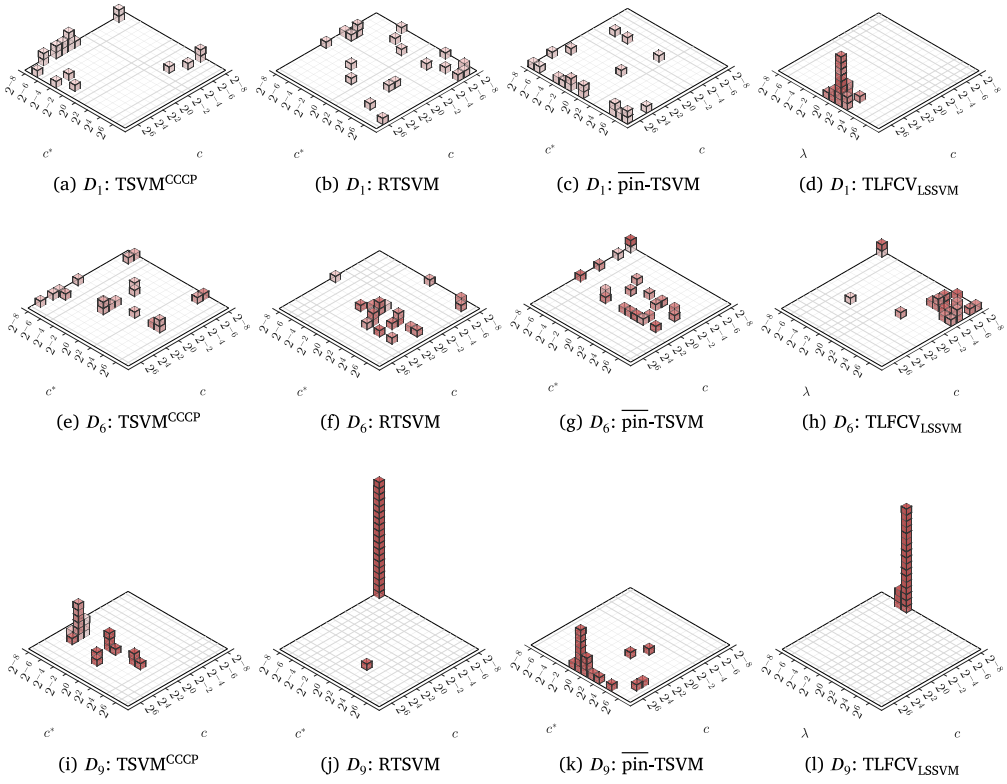
$$\gamma^* = \arg\min_{\gamma\in\Pi_2} \frac{1}{|\mathbb{N}_l|^2} \sum_{i\in\mathbb{N}_l}\sum_{j\in\mathbb{N}_l} \left|\exp\left(-\gamma\|\mathbf{x}_i-\mathbf{x}_j\|^2\right) - \mathbf{J}_{ij}\right|. \tag{27}$$

Fig. 8 compares the performance on D_1 using the selected ideal kernel against the best kernel found by an exhaustive grid search.

Although the accuracy of the selected ideal kernel is slightly below the grid optimum, the difference (Δ Acc.) is marginal, and the performance trends stay consistent across the other hyperparameters. This indicates that the ideal kernel gives a reliable approximation for the kernel-based methods shown in Fig. 8.

2) *Parameter Distribution*. Ideally, the optimal hyperparameters of a robust transductive model should be insensitive to the random selection of the labeled data. Since the accuracy reported above is the best one over the whole grid, we further examine whether the configuration that attains it stays the same when the labeled samples change. To assess parameter stability, we track the optimal configurations across 20 independent trials. Fig. 9 shows the results on D_1, D_6 and D_9, which differ in both the number of samples and the number of features. The four methods shown here all search over a pair of parameters, while TLFVS$_{LSSVM}$ searches over c alone and is therefore not included. As shown, the optimal parameters of TSVM^{CCCP}, RTSVM, and $\overline{\text{pin}}$-TSVM are widely scattered, revealing a severe sensitivity to the labeled samples. In contrast, TLFCV$_{LSSVM}$ maintains a tightly clustered parameter distribution, which means that its parameters can be reused across trials.

3) *Effect of the Cross-Validation Parameter k*. Table 5 reports the transductive accuracy of TLFCV$_{LSSVM}$ for different k values, together with significance tests against LOO. Following common practice in machine learning (Demšar, 2006), for each dataset a Friedman test first assesses whether the choice of k has any overall effect; each k is then compared



(a) D_1: TSVM^{CCCP} (b) D_1: RTSVM (c) D_1: $\overline{\text{pin}}$-TSVM (d) D_1: TLFCV$_{\text{LSSVM}}$

(e) D_6: TSVM^{CCCP} (f) D_6: RTSVM (g) D_6: $\overline{\text{pin}}$-TSVM (h) D_6: TLFCV$_{\text{LSSVM}}$

(i) D_9: TSVM^{CCCP} (j) D_9: RTSVM (k) D_9: $\overline{\text{pin}}$-TSVM (l) D_9: TLFCV$_{\text{LSSVM}}$

Fig. 9. Distribution of optimal hyperparameter configurations across 20 trials on D_1, D_6 and D_9, three datasets of different sizes, for TSVM^{CCCP}, RTSVM, $\overline{\text{pin}}$-TSVM and TLFCV$_{\text{LSSVM}}$. The height of a bar is the number of the 20 trials in which the corresponding configuration is optimal. The color of each square represents the accuracy in the corresponding trial, with darker shades indicating higher accuracy.

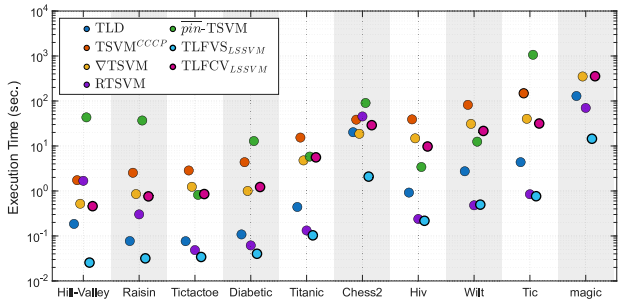
Table 5
Mean accuracy (%) and standard deviation over 20 trials on D_1, D_2 and D_3 for different values of k.

Dataset	Metric	$k=2$	$k=3$	$k=5$	$k=10$	$k=20$	$k=\lvert\mathbb{N}_u\rvert$ (LOO)
D_1	Accuracy	59.74 ± 1.21	61.21 ± 1.25	62.61 ± 1.26	63.18 ± 1.06	63.33 ± 1.03	62.97 ± 0.00
	p-value	0.0004*	0.0005*	0.5479	0.5479	0.2714	–
D_2	Accuracy	86.59 ± 0.40	86.65 ± 0.17	86.68 ± 0.26	86.70 ± 0.22	86.65 ± 0.15	86.70 ± 0.00
	p-value	0.6681	0.9031	1.0000	1.0000	0.5823	–
D_3	Accuracy	72.74 ± 0.58	72.42 ± 0.38	72.36 ± 0.47	74.30 ± 2.38	77.70 ± 4.08	73.35 ± 0.00
	p-value	0.0038*	0.0004*	0.0004*	0.3222	0.0016*	–

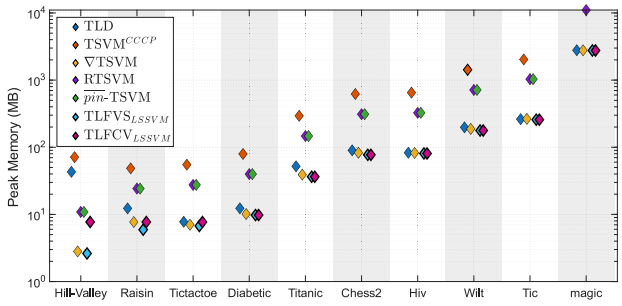
* indicates a statistically significant difference from LOO (Wilcoxon signed-rank test, Holm-corrected, $p < -0.05$).

with LOO using the Wilcoxon signed-rank test with Holm correction (Holm, 1979). On D_1, small values ($k = 2, 3$) fall significantly below LOO, but the gap closes once $k \geq 5$, where the accuracy becomes statistically indistinguishable from LOO. On D_2, the Friedman test detects no significant effect of k at all, indicating that even a minimal k suffices on this dataset. On D_3, small values ($k = 2, 3, 5$) again underperform LOO, while $k = 10$ matches it and $k = 20$ significantly surpasses it, showing that LOO is not necessarily the optimal choice. Overall, a larger k does not necessarily yield a better result. The bound of Theorem 1 does not become tighter as k grows, since the exponent in (A2) carries $|\mathbb{N}_u|$ and not $k|\mathbb{N}_u|$. The cross-validation term here filters candidate labelings rather than estimating the error of a fixed model, and a smaller validation error does not mean the labeling is more reliable. Meanwhile the construction of $\mathbf{H}$ takes k solves and the size of (21) grows with k, and Table 5 shows that $k = 5$ is already indistinguishable from LOO on most of the datasets. We therefore set $k = 5$ by default.

4) *Computational Cost*. We also record the cost of solving each model once. Fig. 10 compares the runtime and peak memory usage on the benchmark datasets under a single parameter setting. TLFVS$_{\text{LSSVM}}$ solves only a single linear system, so it has the shortest runtime. By contrast, the TSVM methods iterate to refine pseudo-labels, which leads to higher memory use and longer runtime. TLFCV$_{\text{LSSVM}}$ keeps a moderate cost, since it solves k linear systems and then the integer program (22). The same pattern appears on the real-world dataset in Table 4.



(a) Runtime



(b) Peak memory

Fig. 10. Comparison of runtime and peak memory usage under a single parameter setting across the benchmark datasets.

6. Conclusion

The validation risk minimization has been proposed for transductive learning in this paper. According to the two existing validation strategies, two learning frameworks TLFVS and TLFCV have been designed and instantiated by the LSSVM, i.e., TLFVS$_{\text{LSSVM}}$ and TLFCV$_{\text{LSSVM}}$. Theoretical analysis has shown that TLFCV is more stable than TLFVS but has a higher computational cost. Additionally, the various integer programming problems in the instantiated TLFVS$_{\text{LSSVM}}$ and TLFCV$_{\text{LSSVM}}$ have been solved efficiently. Experiments on synthetic, benchmark, and real-world datasets have shown that our TLFCV$_{\text{LSSVM}}$ outperforms many existing transductive learning models. Generally, different data splitting and learning machines in framework (6) result in different transductive learning models, and certainly, more complicated bilevel programming problems have to be considered. Therefore, future work includes building transductive models with various data splitting and other learning machines in framework (6). The main difficulty is that a learner without a closed-form solution keeps the inner problem in place. Two routes are worth trying. One is to shrink the space of candidate labelings so that the cost of the search becomes acceptable. The other is to solve the bilevel program through the KKT conditions of the inner problem or implicit differentiation. Designing efficient algorithms for bilevel programming problems and mixed integer programming problems is also of interest.

CRediT authorship contribution statement

Yan Lv: Writing – original draft, Investigation, Formal analysis, Data curation; **Zhen Wang:** Writing – review & editing, Software, Conceptualization; **Lan Bai:** Supervision, Resources, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported in part by National Natural Science Foundation of China (No. 62366035), and in part by Natural Science Foundation of Inner Mongolia Autonomous Region of China (No. 2025MS06008).

References

Adams, W. P., & Sherali, H. D. (1986). A tight linearization and an algorithm for zero-one quadratic programming problems. *Management Science*, *32*(10), 1274–1290.

Akram-Ali-Hammouri, Z., Fernández-Delgado, M., Cernadas, E., & Barro, S. (2021). Fast support vector classification for large-scale problems. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *44*(10), 6184–6195.

Arazo, E., Ortego, D., Albert, P., O'Connor, N. E., & McGuinness, K. (2020). Pseudo-labeling and confirmation bias in deep semi-supervised learning. In *International joint conference on neural networks* (pp. 1–8).

Belkin, M., Niyogi, P., & Sindhwani, V. (2006). Manifold regularization: A geometric framework for learning from labeled and unlabeled examples. *Journal of Machine Learning Research*, *7*(11), 2399–2434.

Bengio, Y., & Grandvalet, Y. (2004). No unbiased estimator of the variance of k-fold cross-validation. *Journal of Machine Learning Research*, *5*(Sep), 1089–1105.

Berenguer, A. D., Kvasnytsia, M., Bossa, M. N., Mukherjee, T., Deligiannis, N., & Sahli, H. (2024). Semi-supervised medical image classification via distance correlation minimization and graph attention regularization. *Medical Image Analysis*, *94*, 103107.

Blondel, M., Berthet, Q., Cuturi, M., Frostig, R., Hoyer, S., Llinares-López, F., Pedregosa, F., & Vert, J.-P. (2022). Efficient and modular implicit differentiation. In *Advances in neural information processing systems* (pp. 5230–5242). (*Vol. 35*).

Boudiaf, M., Ziko, I. M., Rony, J., Dolz, J., Piantanida, P., & Ben Ayed, I. (2020). Information maximization for few-shot learning. In *Advances in neural information processing systems* (pp. 2445–2457). (*Vol. 33*).

Cevikalp, H., & Franc, V. (2017). Large-scale robust transductive support vector machines. *Neurocomputing*, *235*, 199–209.

Chapelle, O., Vapnik, V., & Weston, J. (1999). Transductive inference for estimating values of functions. In *Advances in neural information processing systems* (pp. 421–427). (*Vol. 12*).

Chapelle, O., & Zien, A. (2005). Semi-supervised classification by low density separation. In *International workshop on artificial intelligence and statistics* (pp. 57–64).

Chen, X., Xiao, T., & Balasubramanian, K. (2024). Optimal algorithms for stochastic bilevel optimization under relaxed smoothness conditions. *Journal of Machine Learning Research*, *25*(151), 1–51.

Ciampi, L., Lagani, G., Amato, G., & Falchi, F. (2026). Biologically-inspired semi-supervised semantic segmentation for biomedical imaging. *Neural Networks*, *201*, 108925.

Collobert, R., Sinz, F., Weston, J., Bottou, L., & Joachims, T. (2006). Large scale transductive SVMs. *Journal of Machine Learning Research*, *7*(62), 1687–1712.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, *20*(3), 273–297.

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, *7*, 1–30.

Ding, S., Zhu, Z., & Zhang, X. (2017). An overview on semi-supervised support vector machine. *Neural Computing and Applications*, *28*(5), 969–978.

El-Yaniv, R., & Pechyony, D. (2009). Transductive rademacher complexity and its applications. *Journal of Artificial Intelligence Research*, *35*, 193–234.

Hajihosseinlou, M., Maghsoudi, A., & Ghezelbash, R. (2025). A semi-supervised approach for mineral prospectivity mapping via weighted positive-unlabeled learning and tree-structured Parzen estimator for hyperparameter optimization. *Ore Geology Reviews*, *185*, 106783.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, *6*, 65–70.

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, *2*(5), 359–366.

Huang, S., Wang, S., Qiao, S., Liu, Y., & Liu, T. (2025). SEMI-PLC: A framework for semi-supervised medical images segmentation with pseudo label correction. *Computer Methods and Programs in Biomedicine*, *271*, 109027.

Iscen, A., Tolias, G., Avrithis, Y., & Chum, O. (2019). Label propagation for deep semi-supervised learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 5070–5079).

Joachims, T. (1999). Transductive inference for text classification using support vector machines. In *Proceedings of the international conference on machine learning* (pp. 200–209).

Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International conference on learning representations*.

Kochenberger, G., Hao, J.-K., Glover, F., Lewis, M., Lü, Z., Wang, H., & Wang, Y. (2014). The unconstrained binary quadratic programming problem: A survey. *Journal of Combinatorial Optimization*, *28*(1), 58–81.

Li, Q., Han, Z., & Wu, X. (2018). Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 3538–3545). (*Vol. 32*).

Li, Y., & Zhou, Z. (2014). Towards making unlabeled data never hurt. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *37*(1), 175–188.

Liang, X., Fu, Y., Qu, L., Zhang, P., & Chen, Y. (2023). Prediction of drug side effects with transductive matrix co-completion. *Bioinformatics*, *39*(1), btad006.
Liu, R., Gao, J., Zhang, J., Meng, D., & Lin, Z. (2021). Investigating bi-level optimization for learning and vision from a unified perspective: A survey and beyond. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *44*(12), 10045–10067.
Liu, Y., Lee, J., Park, M., Kim, S., Yang, E., Hwang, S. J., & Yang, Y. (2019). Learning to propagate labels: Transductive propagation network for few-shot learning. In *International conference on learning representations*.
Luo, Z., Farahbakhsh, E., Hore, S., & Müller, R. D. (2026). DEEP-SEAM: An explainable semi-supervised deep learning framework for mineral prospectivity mapping. *Geoscientific Model Development*, *19*(7), 2593–2625.
Oliver, A., Odena, A., Raffel, C. A., Cubuk, E. D., & Goodfellow, I. (2018). Realistic evaluation of deep semi-supervised learning algorithms. In *Advances in neural information processing systems* (pp. 3235–3246).
Petrulionyte, I., Mairal, J., & Arbel, M. (2024). Functional bilevel optimization for machine learning. In *Advances in neural information processing systems* (pp. 14016–14065). (*Vol. 37*).
Pham, H., Dai, Z., Xie, Q., & Le, Q. V. (2021). Meta pseudo labels. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 11557–11568).
Poon, H.-K., Yap, W.-S., Tee, Y.-K., Lee, W.-K., & Goi, B.-M. (2019). Hierarchical gated recurrent neural network with adversarial and virtual adversarial training on text classification. *Neural Networks*, *119*, 299–312.
Raghavachari, M. (1969). On connections between zero-one integer programming and concave programming under linear constraints. *Operations Research*, *17*(4), 680–684.
Shi, Y., Pi, Y., Liu, Z., Zhao, H., & Wang, S. (2025). Information-controlled graph convolutional network for multi-view semi-supervised classification. *Neural Networks*, *184*, 107102.
Shu, J., Xie, Q., Yi, L., Zhao, Q., Zhou, S., Xu, Z., & Meng, D. (2019). Meta-weight-net: Learning an explicit mapping for sample weighting. In *Advances in neural information processing systems* (pp. 1908–1919).
Singla, M., Ghosh, D., & Shukla, K. K. (2021). Pin-TSVM: A robust transductive support vector machine and its application to the detection of COVID-19 infected patients. *Neural Processing Letters*, *53*(6), 3981–4010.
Suykens, J. A. K., & Vandewalle, J. (1999). Least squares support vector machine classifiers. *Neural Processing Letters*, *9*(3), 293–300.
Vapnik, V. N. (1998). Statistical learning theory. New York: Wiley.
Wang, X., & Liu, J. (2025). Multi-spectral attention and graph smoothness enhancement for generalized node classification. *Neural Networks*, *195*, 108287.
Watters, L. J. (1967). Reduction of integer polynomial programming problems to zero-one linear programming problems. *Operations Research*, *15*(6), 1171–1174.
Wu, Q., Yang, C., Zhao, W., He, Y., Wipf, D., & Yan, J. (2023). DIFFormer: Scalable (graph) transformers induced by energy constrained diffusion. In *International conference on learning representations*.
Wu, Z., Xu, R., Luo, Y., & Shao, H. (2024). A holistic semi-supervised method for imbalanced fault diagnosis of rotational machinery with out-of-distribution samples. *Reliability Engineering & System Safety*, *250*, 110297.
Xu, Y., & Zuo, R. (2024). An interpretable graph attention network for mineral prospectivity mapping. *Mathematical Geosciences*, *56*(2), 169–190.
Yang, X., Song, Z., King, I., & Xu, Z. (2022). A survey on deep semi-supervised learning. *IEEE Transactions on Knowledge and Data Engineering*, *35*(9), 8934–8954.
Yao, L., Mao, C., & Luo, Y. (2019). Graph convolutional networks for text classification. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 7370–7377). (*Vol. 33*).
Zhang, W., Li, X., Ma, H., Luo, Z., & Li, X. (2021). Universal domain adaptation in fault diagnostics with hybrid weighted deep adversarial learning. *IEEE Transactions on Industrial Informatics*, *17*(12), 7957–7967.
Zhao, Z., Xu, Y., Zhang, J., Zhao, R., Chen, Z., & Jiao, Y. (2024). A semi-supervised Gaussian mixture variational autoencoder method for few-shot fine-grained fault diagnosis. *Neural Networks*, *178*, 106482.